

Ödev 16: Bir Log Aracı

Bir dosyayı düzgün okuyan, bir tane güvenle yazan ve yol geçişi denemesini reddeden bir şey kurun.

Oluşturduğunuz her dosya program çıkmadan önce silinmelidir. `Files.createTempDirectory(...)` içinde çalışın ve bir `finally` bloğunda temizleyin. Geride dosya bırakmak bu ödevi geçersiz kılar.

Birinci Bölüm: Akışla Okuyun, Yutmayın

`LogTool.java` dosyasını klasik biçimde oluşturun.

Şu biçimde 1.000 satırlık bir log üretin:

```
ERROR request=0 ms=10
INFO request=1 ms=47
```

Sonra dört soruyu, **her biri kendi** `Files.lines` çağrısıyla cevaplayın:

1. Her seviyeden kaç satır var, seviye adına göre alfabetik
2. İlk `ERROR` satırı
3. En yüksek `ms=` değerine sahip satır
4. Tüm `ms=` değerleri üzerinde özet istatistikler

O stream'lerin hepsi kapatılmalı. try-with-resources olmasaydı neyin sızacağını ve bu metotta kaç kez sızacağını bir yorumda yazın.

İkinci soru için, dosya boyutu ne olursa olsun `readAllLines` metodunun asla yapamayacağı neyi `findFirst` metodunun yaptığını bir yoruma ekleyin.

İkinci Bölüm: Filtrelenmiş Bir Kopya Yazın

Yalnızca `ERROR` satırlarını ikinci bir dosyaya yazın, **iki kaynaklı tek bir try-with-resources** içinde okuyup yazarak.

Bir sorunla karşılaşacaksınız: `forEach` bir `Consumer` alır, ve `write` denetlenen bir `IOException` fırlatır. Çözün, ve genel problemi isimlendirip standart çözümün özgün başarısızlığı neden koruduğunu bir yorumda yazın.

Sonra:

- Sona bir satır ekleyin, ve eklemenin neden açık bir seçenek gerektirdiğini yorumlayın
- İç içe bir dizin oluşturup dosyayı `move` ile içine taşıyın
- Özgününün gittiğini gösterin

Üçüncü Bölüm: Yol Doğrulayıcı

Bir `uploads` dizini oluşturun. Kullanıcıdan gelen bir dosya adını alıp izin verilip verilmediğine karar veren bir metot yazın.

Bu beşinin hepsiyle test edin:

Girdi	Olması gereken
<code>photo.jpg</code>	izin verilir
<code>nested/../photo.jpg</code>	izin verilir
<code>../../etc/passwd</code>	reddedilir
<code>/etc/passwd</code>	reddedilir
<code>..</code>	reddedilir

Dördüncüsü ilginç olanı. `resolve` metodunun mutlak bir argümanla ne yaptığını bulun ve denetiminizin onu yine de yakaladığından emin olun.

Sonra String Tuzağını Kanıtlayın

`uploads--evil` adında kardeş bir dizin ve içinde bir dosya oluşturun.

O dosyanın `String.startsWith` ve `Path.startsWith` metotlarına göre "uploads içinde" olup olmadığını yazdırın. Anlaşamayacaklar.

Nedenini, `Path` metodunun `String` metodunun karşılaştırmadığı neyi karşılaştırdığını ve bunun neden bir merak değil gerçek bir atlatma olduğunu bir yorumda açıklayın.

Dördüncü Bölüm: Bir Şey Söyleyen İstisnalar

Şunların her birini tetikleyip yakalayın ve dosya adını zaten elinizde olan bir değişkenden değil istisnanın kendisinden yazdırın:

- `NoSuchFileException`
- `FileAlreadyExistsException`
- `DirectoryNotEmptyException`

Sonra **eski** `java.io.File` API'si üzerinden eksik bir dosyada `delete()` çağırın ve ne döndürdüğünü yazdırın.

Şunları kapsayan bir yorum yazın:

- Eski dönüş değerinin size ne söylediği ve neyi söyleyemediği
- Önce** `exists()` denetlemenin neden çözüm olmadığı. Yarışı isimlendirin ve istisnanın neden korunmuş görünen bir satırdan yine de gelebileceğini söyleyin

Kabul Kriterleri

- ☐ `java LogTool.java` ile çalışıyor
- ☐ Geçici dizin siliniyor ve program bunu kanıtlıyor
- ☐ Dört ayrı `Files.lines` çağrısı, her biri try-with-resources içinde
- ☐ Bir yorum onlarsız neyin ve kaç kez sızacağını belirtiyor
- ☐ Bir yorum `findFirst` metodunun `readAllLines` üzerine ne kattığını açıklıyor
- ☐ Filtrelenmiş kopya tek try-with-resources içinde iki kaynak kullanıyor
- ☐ Lambda içindeki denetlenen istisna problemi çözülmüş ve açıklanmış
- ☐ Ekleme açık bir dosya açma seçeneği kullanıyor, nedenini söyleyen yorumla
- ☐ Beş doğrulayıcı durumunun hepsi doğru kararı veriyor
- ☐ `String.startsWith` ve `Path.startsWith` anlaşmazlık içinde gösteriliyor
- ☐ Açıklama `Path` metodunun bunun yerine neyi karşılaştırdığını söylüyor
- ☐ Üç dosya istisnası da yakalanıyor ve dosya adları istisnadan yazdırılıyor
- ☐ Bir yorum TOCTOU yarışını ve `exists()` metodunun onu neden çözmediğini isimlendiriyor

İleri Seviye

Filtrelenmiş kopyayı **atomik** yapın: aynı dizinde geçici bir dosyaya yazın, sonra `StandardCopyOption.ATOMIC_MOVE` ile yerine taşıyın.

Sonra atomik olmayan bir yazma sırasında hedef dosyayı okuyan birinin ne gözlemleyebileceğini ve taşımanın atomik olması için "aynı dizin" koşulunun neden önemli olduğunu bir yorumda yazın.

İpucu, `/etc/passwd` elinizden kaçıyor

Normalleştirmeden önce `base.resolve("/etc/passwd")` çağrısının ne döndürdüğüne bakın. Mutlak bir argüman tabana eklenmez, onu tamamen değiştirir, dolayısıyla sonucun uploads dizininizle hiçbir ilgisi kalmaz. Bunu yakalaması gereken sizin kapsama denetiminiz, ki denetimin `resolve` metoduna güvenmek yerine var olma sebebi tam olarak budur.